

Lecture Unit 8.3

More OpenMP work sharing constructs

Work sharing constructs

- OpenMP provides a number of work sharing constructs which describe how to distribute work in a parallel region
- Previously we saw how to use `parallel for` to parallelise loops
- Two other work sharing constructs are:
 - `sections`
 - `single`

Section construct

- Used when there are two or more sections of code which can be executed concurrently

```
#pragma omp sections
{
#pragma omp section
    do_work();

#pragma omp section
    do_other_work();
}
```

sections.c

```
#include <stdio.h>
#include <math.h>

int main(){

    const int N=1e7;
    float x, y;

    #pragma omp parallel sections default(shared) private(x,y)
    {

        #pragma omp section
        {
            FILE *outfile1;
            outfile1 = fopen("exp.dat", "w");
            for (int i=0; i<N; i++){
                x = ( (float) i) / ( (float) N);
                y = exp(x);
                fprintf(outfile1, "%f %f\n", x, y);
            }
            fclose(outfile1);
        }

        #pragma omp section
        {
            FILE *outfile2;
            outfile2 = fopen("tanh.dat", "w");
            for (int i=0; i<N; i++){
                x = ( (float) i) / ( (float) N);
                y = tanh(x);
                fprintf(outfile2, "%f %f\n", x, y);
            }
            fclose(outfile2);
        }

    } // omp parallel sections

    return 0;
}
```

x and y are private to each section

The sections are executed concurrently. The loops are not parallelised so the values are written in the correct order

Single construct

- In parallel region work which is not parallelised is repeated by all threads
- The **single** construct ensures only one thread executes a block of code inside a parallel region but it can be any thread

```
#pragma omp single [(clause1, ...)]
```

- The **single** construct was used in the Mandelbrot program in the workshop last week

Single construct

```
/* Say how many threads we are using. This needs to be done from within a parallel region */
#ifdef _OPENMP
#pragma omp parallel ← Start a parallel region
{
    /* The single pragma means that only one thread writes this message */
    #pragma omp single
    printf("Starting calculation on %d threads \n", omp_get_num_threads());
}
#else
    printf("Starting serial calculation\n");
#endif
```

Starting calculation on 2 threads

```
real    0m53.176s
user    1m31.009s
sys      0m0.077s
```

Starting calculation on 4 threads

```
real    0m25.532s
user    1m31.831s
sys      0m0.071s
```

Starting calculation on 8 threads

```
real    0m14.119s
user    1m39.920s
sys      0m0.071s
```

Starting calculation on 12 threads

```
real    0m9.945s
user    1m46.586s
sys      0m0.093s
```

Starting calculation on 16 threads

```
real    0m7.735s
user    1m49.931s
sys      0m0.062s
```

Only one thread will execute this part but we don't know which one

Only one thread calls `printf`

Unit summary

- In this unit we have seen two additional work sharing constructs in OpenMP:
 - `sections` can be used when two or more sections
 - `single` is used when only one thread needs to of a program can run concurrently execute a block of code in a parallel region
- `parallel for`, `sections` and `single` were the only work sharing constructs in C before OpenMP 3.0